Broker API Integration Requirements

Document Version: 1.0
Date: February 2026
Purpose: Technical specification for integrating a broker's internal system with our Financial Management & Chatbot Platform

---

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Core Entities & Data Models
4. Required API Endpoints
5. Response Format Specifications
6. Chatbot Integration Requirements
7. Data Requirements by Feature
8. Security & Access Control
9. Implementation Guidelines
10. Testing & Validation

Executive Summary

Your broker system must provide REST API endpoints that expose your internal data in a standardized format. This platform will query your API to retrieve:

- **Client Information** - Account details, CDS accounts, profile data
- **Trading Data** - Trades, orders, positions, transaction history
- **Stock Information** - Price data, company profiles, market data
- **Financial Data** - Payments, receipts, fees, balance information
- **Company Data** - Broker company information, regulatory details

The chatbot functionality depends on being able to query this data in real-time and provide natural language interface for your brokers and their clients.

Architecture Overview

Integration Flow

```
Your Broker System
    ↓
    ↓ (REST API)
    ↓
Our Platform APIs
    ↓
```

```
    ├─ Main Query Engine (for reports, dashboards)
    ├─ Chatbot Engine (for conversational queries)
    └─ Mobile/Web Applications
```

Key Principles

1. **READ-ONLY by Default** - All critical endpoints should be GET (read-only)
2. **Stateless API** - Each request should be independent
3. **JSON Format** - All responses must be in JSON
4. **Pagination** - Provide pagination for large datasets
5. **Filtering & Search** - Support column-based filtering and text search
6. **Error Handling** - Consistent error response format

Core Entities & Data Models

1. CLIENTS Entity

Description: Information about your brokerage clients/parties

Required Fields:

```
{
  "id": "integer (unique identifier)",
  "cds_account": "string (unique CDS account number)",
  "client_name": "string (full name or company name)",
  "email": "string (primary email)",
  "phone": "string (phone number)",
  "client_type": "string (Individual, Corporate, Institution, etc.)",
  "account_status": "string (Active, Inactive, Suspended, Closed)",
  "registration_date": "date (YYYY-MM-DD)",
  "country": "string (country code or name)",
  "address": "string (postal address)",
  "contact_person": "string (primary contact)",
  "tax_id": "string (TIN or equivalent)",
  "kyc_status": "string (Compliant, Pending, Non-Compliant)",
  "created_at": "datetime (ISO 8601)",
  "updated_at": "datetime (ISO 8601)"
}
```

Minimum Required Fields:

- id, cds_account, client_name, email, phone, account_status

Unique Identifiers:

- `id` (internal)
- `cds_account` (external, used for cross-system integration)

---

2. TRADES/TRANSACTIONS Entity

Description: Individual trades executed by or for clients

Required Fields:

```
{
  "id": "integer (unique trade identifier)",
  "trade_number": "string (readable trade reference, e.g., TR-001)",
  "trade_date": "date (YYYY-MM-DD - when trade was executed)",
  "settlement_date": "date (YYYY-MM-DD - when trade settles)",
  "client_id": "integer (reference to clients.id)",
  "cds_account": "string (client's CDS account)",
  "symbol": "string (stock/security symbol)",
  "security_name": "string (full name of security)",
  "security_type": "string (Equity, Bond, ETF, Shares, etc.)",
  "trade_type": "string (Buy, Sell)",
  "quantity": "decimal (number of units/shares)",
  "unit_price": "decimal (price per unit)",
  "gross_amount": "decimal (quantity × unit_price)",
  "broker_commission": "decimal (commission charged)",
  "net_amount": "decimal (gross_amount - broker_commission)",
  "counter_party": "string (name of other party or broker)",
  "exchange": "string (exchange name, e.g., DSE, NSE)",
  "settlement_status": "string (Pending, Settled, Failed, Cancelled)",
  "status": "string (Completed, Pending, Rejected, Cancelled)",
  "notes": "string (additional notes)",
  "created_at": "datetime (ISO 8601)",
  "updated_at": "datetime (ISO 8601)"
}
```

Minimum Required Fields:

- id, trade_date, client_id, symbol, trade_type, quantity, unit_price, status

Key Relationships:

- Must link to clients via `client_id` or `cds_account`
- Must link to companies/stock data via `symbol`

---

3. COMPANIES/SECURITIES Entity

Description: Stock symbols, companies, and tradeable securities

Required Fields:

```
{
  "id": "integer (unique identifier)",
  "symbol": "string (unique stock ticker, e.g., CRDB, TBL)",
  "company_name": "string (full legal company name)",
```

```
  "business_type": "string (Bank, Insurance, Retail, Manufacturing, etc.)",
  "sector": "string (Finance, Energy, Healthcare, etc.)",
  "market_cap": "decimal (market capitalization in TZS/USD)",
  "currency": "string (TZS, USD, etc.)",
  "exchange": "string (exchange listing code)",
  "country": "string (country of registration)",
  "registration_number": "string (company registration)",
  "email": "string (company email)",
  "website": "string (company website URL)",
  "status": "string (Listed, Delisted, Suspended, etc.)",
  "listing_date": "date (when listed on exchange)",
  "industry": "string (industry classification)",
  "created_at": "datetime (ISO 8601)",
  "updated_at": "datetime (ISO 8601)"
}
```

Minimum Required Fields:

- id, symbol, company_name, exchange

Unique Identifiers:

- `symbol` (stock ticker)
- `id` (internal reference)

---

4. STOCK PRICES Entity

Description: Historical and current stock price data

Required Fields:

```
{
  "id": "integer (unique identifier)",
  "symbol": "string (stock ticker)",
  "trading_date": "date (YYYY-MM-DD)",
  "opening_price": "decimal (price at market open)",
  "closing_price": "decimal (price at market close)",
  "high_price": "decimal (highest price in the day)",
  "low_price": "decimal (lowest price in the day)",
  "volume": "integer (number of shares traded)",
  "traded_value": "decimal (total monetary value traded)",
  "price_change": "decimal (closing_price - opening_price)",
  "price_change_percent": "decimal (percentage change)",
  "last_updated": "datetime (ISO 8601 - last price update time)",
  "created_at": "datetime (ISO 8601)"
}
```

Minimum Required Fields:

- symbol, trading_date, closing_price, volume

Key Notes:

- Must include at least the latest day's data
- Should include historical data for at least 1 year if available
- Price change calculations should be accurate for technical analysis

5. FEES & COMMISSIONS Entity

Description: Fee configurations and commission structures

Required Fields:

```
{
  "id": "integer (unique identifier)",
  "fee_type": "string (Commission, Service Fee, Listing Fee, etc.)",
  "description": "string (fee description)",
  "rate": "decimal (percentage or fixed amount)",
  "rate_type": "string (Percentage, Fixed Amount)",
  "applies_to": "string (Equity, Bond, ETF, All, etc.)",
  "min_amount": "decimal (minimum fee amount)",
  "max_amount": "decimal (maximum fee amount)",
  "status": "string (Active, Inactive)",
  "effective_from": "date (YYYY-MM-DD)",
  "effective_to": "date (YYYY-MM-DD - null if ongoing)",
  "created_at": "datetime (ISO 8601)",
  "updated_at": "datetime (ISO 8601)"
}
```

Minimum Required Fields:

- id, fee_type, rate, status

---

6. CLIENT BALANCES/ACCOUNTS Entity

Description: Client account balances, holdings, and portfolio data

Required Fields:

```
{
  "id": "integer (unique identifier)",
  "client_id": "integer (reference to clients.id)",
  "cds_account": "string (CDS account number)",
  "account_type": "string (Cash Account, Margin Account, etc.)",
  "cash_balance": "decimal (available cash)",
  "securities_value": "decimal (value of held securities)",
  "total_value": "decimal (cash_balance + securities_value)",
  "margin_available": "decimal (available margin if applicable)",
  "margin_used": "decimal (used margin)",
  "total_charges": "decimal (accumulated fees/charges)",
```

```
  "last_trade_date": "date (date of last trade)",
  "account_status": "string (Active, Suspended, Closed)",
  "as_of_date": "date (balance as of this date)",
  "created_at": "datetime (ISO 8601)",
  "updated_at": "datetime (ISO 8601)"
}
```

Minimum Required Fields:

- id, client_id, cash_balance, securities_value, as_of_date

---

7. ORDERS/PENDING TRANSACTIONS Entity

Description: Pending orders and pending transactions

Required Fields:

```
{
  "id": "integer (unique identifier)",
  "order_number": "string (readable order reference)",
  "client_id": "integer (reference to clients.id)",
  "order_date": "date (YYYY-MM-DD - when order was placed)",
  "symbol": "string (stock symbol)",
  "order_type": "string (Buy, Sell, Short Sell, etc.)",
  "quantity": "integer (number of units)",
  "price_type": "string (Market, Limit, Stop, etc.)",
  "limit_price": "decimal (limit price if applicable)",
  "order_status": "string (Open, Pending, Partial, Filled, Cancelled, Rejected)",
  "filled_quantity": "integer (quantity already filled)",
  "remaining_quantity": "integer (quantity still pending)",
  "average_price": "decimal (average price of filled portion)",
  "order_expiry": "date (when order expires)",
  "notes": "string (additional notes)",
  "created_at": "datetime (ISO 8601)",
  "updated_at": "datetime (ISO 8601)"
}
```

Minimum Required Fields:

- id, client_id, symbol, order_type, quantity, order_status

Required API Endpoints

Your API must provide the following endpoints to work with our platform:

Endpoint Pattern

All endpoints follow this general pattern:

```
GET /api/v1/{resource}
GET /api/v1/{resource}/{id}
POST /api/v1/{resource} (if write operations supported)
PUT /api/v1/{resource}/{id} (if update operations supported)
```

1. Clients Endpoint

Retrieve Client Information

```
GET /api/v1/clients
GET /api/v1/clients/{id}
GET /api/v1/clients?cds_account={account}
```

Query Parameters:

- `page` (integer) - Page number, default: 1
- `limit` (integer) - Records per page, default: 50, max: 500
- `search` (string) - Search by name, email, phone, or CDS account
- `status` (string) - Filter by account_status (Active, Inactive, etc.)
- `sort_by` (string) - Column to sort by, default: id
- `sort_order` (string) - ASC or DESC, default: ASC
- `client_type` (string) - Filter by type (Individual, Corporate)
- `kyc_status` (string) - Filter by kyc_status

Response Example:

```
{
  "success": true,
  "status_code": 200,
  "timestamp": "2026-02-27T10:30:00Z",
  "message": "Clients retrieved successfully",
  "data": {
    "records": [
      {
        "id": 1,
        "cds_account": "CDS123456",
        "client_name": "John Doe",
        "email": "john@example.com",
        "phone": "+255 742 123456",
        "client_type": "Individual",
        "account_status": "Active",
        "registration_date": "2025-01-15",
        "kyc_status": "Compliant",
        "created_at": "2025-01-15T08:00:00Z"
      }
    ],
    "pagination": {
```

```
      "total_records": 1500,
      "current_page": 1,
      "page_size": 50,
      "total_pages": 30
    }
  }
}
```

---

2. Trades Endpoint

Retrieve Trade/Transaction Data

```
GET /api/v1/trades
GET /api/v1/trades/{id}
GET /api/v1/trades?client_id={id}
GET /api/v1/trades?symbol={symbol}
GET /api/v1/trades?cds_account={account}
```

Query Parameters:

- `page` (integer) - Page number
- `limit` (integer) - Records per page, max: 500
- `search` (string) - Search in trade number, symbol, client name
- `client_id` (integer) - Filter by client
- `cds_account` (string) - Filter by CDS account
- `symbol` (string) - Filter by stock symbol
- `status` (string) - Filter by trade status
- `trade_type` (string) - Filter by Buy/Sell
- `from_date` (date) - From trade date (YYYY-MM-DD)
- `to_date` (date) - To trade date (YYYY-MM-DD)
- `sort_by` (string) - Column to sort
- `sort_order` (string) - ASC or DESC

Response Example:

```
{
  "success": true,
  "status_code": 200,
  "timestamp": "2026-02-27T10:30:00Z",
  "message": "Trades retrieved successfully",
  "data": {
    "records": [
      {
        "id": 1001,
        "trade_number": "TR-001",
        "trade_date": "2026-02-27",
        "settlement_date": "2026-03-01",
```

```
          "client_id": 1,
          "cds_account": "CDS123456",
          "symbol": "CRDB",
          "security_name": "Crescent Refineries Ltd",
          "security_type": "Equity",
          "trade_type": "Buy",
          "quantity": 1000,
          "unit_price": 45.50,
          "gross_amount": 45500.00,
          "broker_commission": 455.00,
          "net_amount": 45045.00,
          "exchange": "DSE",
          "settlement_status": "Settled",
          "status": "Completed",
          "created_at": "2026-02-27T09:15:00Z"
        }
      ],
      "pagination": {
        "total_records": 15000,
        "current_page": 1,
        "page_size": 50,
        "total_pages": 300
      }
    }
  }
```

---

3. Companies Endpoint

Retrieve Company/Security Information

```
GET /api/v1/companies
GET /api/v1/companies/{id}
GET /api/v1/companies?symbol={symbol}
```

Query Parameters:

- `page` (integer) - Page number
- `limit` (integer) - Records per page, max: 500
- `search` (string) - Search by company name, symbol, or registration
- `symbol` (string) - Filter by stock symbol
- `sector` (string) - Filter by sector
- `status` (string) - Filter by status (Listed, Delisted, etc.)
- `exchange` (string) - Filter by exchange

Response Example:

```
{
  "success": true,
```

```
  "status_code": 200,
  "timestamp": "2026-02-27T10:30:00Z",
  "message": "Companies retrieved successfully",
  "data": {
    "records": [
      {
        "id": 1,
        "symbol": "CRDB",
        "company_name": "Crescent Refineries Ltd",
        "business_type": "Retail/Trading",
        "sector": "Energy",
        "market_cap": 50000000000.00,
        "currency": "TZS",
        "exchange": "DSE",
        "country": "Tanzania",
        "status": "Listed",
        "listing_date": "2015-06-01",
        "website": "www.crdb.co.tz"
      }
    ],
    "pagination": {
      "total_records": 60,
      "current_page": 1,
      "page_size": 50,
      "total_pages": 2
    }
  }
}
```

---

4. Stock Prices Endpoint

Retrieve Stock Price Data

```
GET /api/v1/stock_prices
GET /api/v1/stock_prices?symbol={symbol}
GET /api/v1/stock_prices?symbol={symbol}&date={date}
GET /api/v1/stock_prices?symbol={symbol}&from_date={date}&to_date={date}
GET /api/v1/stock_prices?symbol={symbol}&days={days}
```

Query Parameters:

- `symbol` (string) - Stock symbol (required or return today's data for all symbols)
- `date` (date) - Specific date (YYYY-MM-DD)
- `from_date` (date) - Start date for range
- `to_date` (date) - End date for range
- `days` (integer) - Number of historical days (1-365)
- `limit` (integer) - Max records to return, default: 100, max: 1000

Response Example:

```json
{
  "success": true,
  "status_code": 200,
  "timestamp": "2026-02-27T16:00:00Z",
  "message": "Stock prices retrieved successfully",
  "data": {
    "records": [
      {
        "id": 5001,
        "symbol": "CRDB",
        "trading_date": "2026-02-27",
        "opening_price": 45.00,
        "closing_price": 45.50,
        "high_price": 46.00,
        "low_price": 44.80,
        "volume": 1250000,
        "traded_value": 56875000.00,
        "price_change": 0.50,
        "price_change_percent": 1.11,
        "last_updated": "2026-02-27T16:00:00Z"
      }
    ],
    "meta": {
      "symbol": "CRDB",
      "records_returned": 1,
      "date_range": {
        "from": "2026-02-27",
        "to": "2026-02-27"
      }
    }
  }
}
```

---

5. Account Balance Endpoint

Retrieve Client Account Balances

```
GET /api/v1/account_balances
GET /api/v1/account_balances/{id}
GET /api/v1/account_balances?client_id={id}
GET /api/v1/account_balances?cds_account={account}
```

Query Parameters:

- `client_id` (integer) - Filter by client
- `cds_account` (string) - Filter by CDS account
- `sort_by` (string) - Column to sort
- `as_of_date` (date) - Get balance as of specific date

Response Example:

```
{
  "success": true,
  "status_code": 200,
  "message": "Account balance retrieved successfully",
  "data": {
    "records": [
      {
        "id": 1,
        "client_id": 1,
        "cds_account": "CDS123456",
        "account_type": "Cash Account",
        "cash_balance": 1500000.00,
        "securities_value": 2500000.00,
        "total_value": 4000000.00,
        "account_status": "Active",
        "as_of_date": "2026-02-27"
      }
    ]
  }
}
```

---

6. Fees Configuration Endpoint

Retrieve Fee Structures

```
GET /api/v1/fees
GET /api/v1/fees/{id}
GET /api/v1/fees?type={fee_type}
```

Query Parameters:

- `type` (string) - Filter by fee type
- `applies_to` (string) - Filter by applies_to
- `status` (string) - Filter by status
- `effective_date` (date) - Get fees effective as of date

Response Example:

```
{
  "success": true,
  "status_code": 200,
  "message": "Fees retrieved successfully",
  "data": {
    "records": [
      {
```

```
          "id": 1,
          "fee_type": "Commission",
          "description": "Standard brokerage commission for equities",
          "rate": 1.0,
          "rate_type": "Percentage",
          "applies_to": "Equity",
          "min_amount": 100000.00,
          "max_amount": null,
          "status": "Active",
          "effective_from": "2026-01-01"
        }
      ]
    }
  }
```

---

7. Orders Endpoint (Pending/Open Orders)

Retrieve Pending Orders

```
GET /api/v1/orders
GET /api/v1/orders/{id}
GET /api/v1/orders?client_id={id}
GET /api/v1/orders?status=open
```

Query Parameters:

- `client_id` (integer) - Filter by client
- `status` (string) - Filter by status (Open, Pending, Filled, etc.)
- `symbol` (string) - Filter by stock symbol
- `search` (string) - Search order

Response Example:

```
{
  "success": true,
  "status_code": 200,
  "message": "Orders retrieved successfully",
  "data": {
    "records": [
      {
        "id": 3001,
        "order_number": "ORD-001",
        "client_id": 1,
        "order_date": "2026-02-27",
        "symbol": "CRDB",
        "order_type": "Buy",
        "quantity": 500,
        "price_type": "Limit",
```

```
          "limit_price": 44.00,
          "order_status": "Open",
          "filled_quantity": 0,
          "remaining_quantity": 500,
          "created_at": "2026-02-27T09:30:00Z"
        }
      ]
    }
  }
```

Response Format Specifications

Standard Success Response

All successful responses should follow this format:

```
{
  "success": true,
  "status_code": 200,
  "timestamp": "2026-02-27T10:30:00Z",
  "message": "Descriptive success message",
  "data": {
    "records": [],
    "pagination": {
      "total_records": 0,
      "current_page": 1,
      "page_size": 50,
      "total_pages": 0
    }
  }
}
```

Pagination Structure

For endpoints returning multiple records:

```
{
  "pagination": {
    "total_records": 1500,
    "current_page": 1,
    "page_size": 50,
    "total_pages": 30,
    "has_next": true,
    "has_previous": false
  }
}
```

Error Response Format

All errors should follow this format:

```
{
  "success": false,
  "status_code": 400,
  "timestamp": "2026-02-27T10:30:00Z",
  "message": "Brief error message",
  "error": {
    "code": "ERROR_CODE",
    "details": "Detailed error explanation",
    "hint": "Suggestion for fixing"
  }
}
```

Common HTTP Status Codes

Code	Meaning	Usage
200	OK	Successful GET/POST/PUT request
201	Created	Resource successfully created (POST)
204	No Content	Successful DELETE
400	Bad Request	Invalid parameters or request format
401	Unauthorized	Authentication required
403	Forbidden	Access denied
404	Not Found	Resource not found
405	Method Not Allowed	HTTP method not supported
409	Conflict	Request conflicts with current state
500	Server Error	Internal server error
503	Service Unavailable	API temporarily unavailable

Chatbot Integration Requirements

The chatbot requires the ability to answer common broker operations questions. Below are the essential data patterns the chatbot needs:

1. Client Lookup Queries

Chatbot: "Show me client John Doe"

```
GET /api/v1/clients?search=John%20Doe&limit=10
```

Required Response Data:

- Client ID, name, email, phone, account status
- CDS account number (for linking to trades/balances)

---

2. Trading History Queries

Chatbot: "What trades did client CDS123456 make this month?"

```
GET /api/v1/trades?cds_account=CDS123456&from_date=2026-02-01&to_date=2026-02-28&limit=100
```

Required Response Data:

- Trade date, symbol, quantity, price, type (buy/sell)
- Settlement status

---

3. Stock Price Queries

Chatbot: "What's the current price of CRDB?"

```
GET /api/v1/stock_prices?symbol=CRDB
GET /api/v1/stock_prices?symbol=CRDB&days=30
```

Required Response Data:

- Current/today's closing price
- Price change and percentage
- Historical data for trends (if requested)

---

4. Account Balance Queries

Chatbot: "What's the balance for client John Doe?"

```
GET /api/v1/clients?search=John%20Doe&limit=1
  [returns: client_id]
GET /api/v1/account_balances?client_id={id}
```

Required Response Data:

- Cash balance, securities value
- Total account value

- Account status

---

5. Fee Queries

Chatbot: "What are the current commission rates?"

```
GET /api/v1/fees?status=Active
```

Required Response Data:

- Fee type, rate, min/max amounts
- What it applies to
- Effective dates

---

6. Complex Multi-Step Queries

Chatbot: "Show me top performers this month"

Sequence of calls:

```
1. GET /api/v1/trades?from_date=2026-02-01&to_date=2026-02-28&limit=500
2. Aggregate by client
3. For top clients: GET /api/v1/clients?id={id}
4. Return sorted summary
```

Required Response Data:

- Per-client trade volumes and values
- Commission earned
- Client details

Data Requirements by Feature

Dashboard Reports

The dashboard requires:

- **Real-time Account Balances** - Current cash/securities/total values
- **Trade Summaries** - Today's trades, volume, value
- **Client Count** - Active clients, new clients
- **Fee/Commission Summary** - Today's collections, pending payments

Chatbot Conversations

The chatbot requires:

- **Search Capabilities** - Full-text search across client names, symbols, trade numbers
- **Date Filtering** - Range queries, relative dates (today, this month, last quarter)
- **Relationship Queries** - Link clients to trades to companies to prices
- **Aggregation Support** - Count trades by status, sum volumes, average prices
- **Performance Metrics** - Volume traded, fees earned, top performers

Mobile/Web Applications

Applications require:

- **Low-latency Access** - Responses in < 1 second for pagination
- **Bulk Export** - Ability to retrieve 500+ records at once
- **Filter Combinations** - Multiple filters simultaneously (status + symbol + date range)
- **Sorting Options** - Any column sortable in either direction

---

Security & Access Control

Authentication Requirements

Implement one of:

1. **API Key Authentication**

   ```
   GET /api/v1/clients?api_key={key}
   Headers: X-API-Key: {key}
   ```

2. **OAuth 2.0**

   - Standard implementation with bearer tokens
   - Token expiration and refresh capability

3. **Basic Authentication** (not recommended for production)

   ```
   Authorization: Basic base64(username:password)
   ```

Authorization Requirements

- All client data access should be role-based
- Ensure users can only access allowed clients/data
- Log all API accesses for audit trails

CORS Headers

Support CORS for browser-based applications:

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, OPTIONS, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization, X-API-Key
```

Rate Limiting

Implement rate limiting:

- **Recommended:** 1000 requests/minute per API key
- **Burst:** Allow up to 100 requests/second
- Return `429 Too Many Requests` when exceeded

Data Privacy

- Never include sensitive data (like passwords) in responses
- Mask sensitive fields (last 4 digits of accounts, etc.)
- Implement field-level access control if needed
- Audit logging for sensitive data access

Implementation Guidelines

Phase 1: Core Implementation (Weeks 1-2)

Priority 1 - Essential Endpoints:

1. Clients endpoint (GET all, GET by ID, search)
2. Trades endpoint (GET all, GET by client, by date range)
3. Companies endpoint (GET all, GET by symbol)
4. Stock Prices endpoint (GET current, GET historical)

Success Criteria:

- All 4 endpoints functional
- Pagination working
- Date filtering working
- Search functionality operational

Phase 2: Extended Data (Weeks 3-4)

Priority 2 - Supporting Endpoints:

1. Employees endpoint
2. Payments endpoint
3. Account Balance endpoint
4. Fees endpoint

Success Criteria:

- All 4 endpoints functional

- Proper relationships between data
- Filters and sorting working

Phase 3: Optimization (Weeks 5-6)

Priority 3 - Advanced Features:

1. Orders/Pending transactions endpoint
2. Performance optimization (caching, indexing)
3. Advanced filtering combinations
4. Bulk operations

Testing & Validation

Test Cases for Each Endpoint

Test 1: Basic Retrieval

```
GET /api/v1/{resource}
Expected: Returns list with pagination
```

Test 2: ID-Based Retrieval

```
GET /api/v1/{resource}/{id}
Expected: Returns single record with all fields populated
```

Test 3: Search Function

```
GET /api/v1/{resource}?search={term}
Expected: Returns matching records
```

Test 4: Pagination

```
GET /api/v1/{resource}?page=1&limit=50
Expected: Returns correct number of records, pagination info accurate
```

Test 5: Filtering

```
GET /api/v1/{resource}?status={value}
Expected: Returns only records matching filter
```

Test 6: Sorting

```
GET /api/v1/{resource}?sort_by={column}&sort_order=DESC
Expected: Records returned in correct order
```

Test 7: Date Range

```
GET /api/v1/{resource}?from_date=2026-01-01&to_date=2026-02-27
Expected: Returns records within date range
```

Test 8: Combined Filters

```
GET /api/v1/{resource}?status=active&symbol=CRDB&from_date=2026-01-01
Expected: All filters applied correctly
```

Performance Tests

- Response time < 1000ms for paginated requests
- Response time < 500ms for single record retrieval
- Support for parallel requests without degradation
- Memory usage stable with large result sets

Data Validation

- All dates in ISO 8601 format (YYYY-MM-DD)
- All times include timezone (Z for UTC)
- Decimal numbers with appropriate precision
- No circular references in relationships
- Consistent null/empty value representation

Integration Checklist

- ☐ All required data entities mapped to your system
- ☐ Core endpoints (clients, trades, companies, stock_prices) implemented
- ☐ Pagination implemented (page, limit, total_records, total_pages)
- ☐ Search functionality across all endpoints
- ☐ Filtering by key columns
- ☐ Sorting by any column (ASC/DESC)
- ☐ Date range filtering
- ☐ Error handling with consistent format

- ☐ HTTP status codes correct
- ☐ CORS headers configured
- ☐ Authentication mechanism implemented
- ☐ Rate limiting configured
- ☐ Response times optimized (< 1000ms)
- ☐ API documentation created
- ☐ Test suite completed
- ☐ Load testing completed
- ☐ Security audit completed
- ☐ Monitoring & logging configured
- ☐ Support contacts established

Support & Questions

For questions or clarifications on these requirements:

1. **Technical Support:** Provide system architecture and current API details
2. **Data Mapping:** Clarify how broker's internal fields map to our entities
3. **Custom Fields:** Discuss additional fields needed beyond minimums
4. **Performance:** Discuss expected data volumes and query patterns
5. **Authentication:** Finalize security implementation approach

---

Version History

Version	Date	Changes
1.0	2026-02-27	Initial comprehensive specification

Document Status: Final
Last Updated: February 27, 2026
Classification: Technical Specification